

Chapter 1

Introduction to Compilers

Language Processors • Structure of a Compiler • Phases & Passes

Evolution of Programming Languages • The Science of Building a Compiler

Applications of Compiler Technology

Prepared by Azmari Sultana

BSc in Computer Science and Engineering, BRAC University

Reference: Compilers – Principles, Techniques & Tools (Aho, Lam, Sethi, Ullman)

Table of Contents

1. Introduction to Compilers	3
1.1 What is a Compiler?	3
2. Language Processors	3
2.1 Compiler vs Interpreter	3
2.2 Hybrid Compilers – The Java Model	4
2.3 Complete Language-Processing System	5
3. Structure of a Compiler	6
3.1 Analysis and Synthesis (Front End / Back End)	6
3.2 Phases of a Compiler	7
3.3 Symbol Table	10
3.4 Grouping of Phases into Passes	10
3.5 Compiler-Construction Tools	10
4. Simulation: Translating an Assignment Statement	11
5. Errors in Compilation	12
6. Evolution of Programming Languages	12
6.1 Generations of Languages	13
6.2 Classification of Languages	13
6.3 Impact on Compilers	14
7. The Science of Building a Compiler	14
7.1 Modeling in Compiler Design	14
7.2 The Science of Code Optimization	14
8. Applications of Compiler Technology	15
8.1 Implementation of High-Level Languages	15
8.2 Optimizations for Computer Architectures	16
8.3 Design of New Computer Architectures	17
8.4 Program Translations	18
8.5 Software Productivity Tools	18
9. Extra: Types of Compilers and Interpreters	19

1. Introduction to Compilers

Programmers write code in high-level languages such as C, Java, or Python because these languages are easy for humans to read and write. But a computer's processor only understands machine language (0s and 1s). So, before a program can run, it must be translated into a form the machine can execute. The software that performs this translation is called a **compiler**.

1.1 What is a Compiler?

A **compiler** is a program that reads a program written in a **source language** and translates it into an equivalent program in a **target language**. While translating, it also reports any errors found in the source program.

- **Source language** — the language the programmer writes in (e.g. C, C++, Java, Python).
- **Target language** — the language produced after translation (usually machine code, assembly, or bytecode).

Two Main Functions of a Compiler

- Translate the source program into an equivalent target program.
- Detect and report errors found during translation.

Why Do We Need a Compiler?

A computer cannot directly understand high-level languages, so a compiler converts the high-level program into machine language. Compilation happens **before** execution — the entire source code is translated first, and the resulting executable is run afterward.

Q1. What is a compiler and what are its two main functions?

A: A compiler translates a program from a source language into an equivalent target language program. Its two main functions are: (1) translating the source program, and (2) detecting and reporting errors during translation.

Q2. Differentiate between source language and target language.

A: Source language is the language in which the programmer writes the code (e.g. C, Java). Target language is the language the compiler produces as output (usually machine code, assembly, or bytecode).

2. Language Processors

A **language processor** is software that converts a program from one language into another, or executes it directly. The two main types are the **compiler** and the **interpreter**. Java uses a combination of both — a hybrid compiler.

Q3. What is a language processor?

A: A language processor is a software program that converts a program written in one language into another language, or executes it directly. Compilers and interpreters are the two main types.

2.1 Compiler vs Interpreter

A compiler translates the **whole** source program into a target program before execution. An **interpreter** does not produce a separate target program — instead, it directly executes the operations of the source program on the given input, working statement by statement.

Working steps of an interpreter:

Read one statement -> Translate it -> Execute it -> Move to next statement

Compiler	Interpreter
Translates the entire program at once	Translates and executes one statement at a time
Produces a separate target program	Does not produce a separate target program
Faster execution (already translated)	Slower execution
Errors are shown after compilation	Errors are shown immediately, statement by statement
Runs machine code directly	Executes source code indirectly, statement by statement

A compiled program runs faster because the entire program is already translated into machine code before execution begins. An interpreter gives better error diagnostics because it stops exactly at the statement where the error occurs, making it easy to locate.

Q4. What is the difference between a compiler and an interpreter?

A: A compiler translates the whole program into a target program before it runs, giving faster execution but delayed error reporting. An interpreter translates and executes the program statement by statement without producing a separate target file, giving slower execution but immediate, precise error diagnostics.

Q5. What are the advantages of a compiler over an interpreter, and of an interpreter over a compiler?

A: (a) Compiler advantages — faster execution since code is pre-translated; produces a standalone executable; suitable for large programs; no need to retranslate on every run.

(b) Interpreter advantages — better error diagnostics; easier debugging (statement by statement); immediate execution without producing an executable file.

2.2 Hybrid Compilers – The Java Model

Some languages combine both compilation and interpretation — this is called a **hybrid compiler**. Java is the classic example.

Java source program --(Java compiler)--> Bytecode --(JVM interpreter)--> Output

- **Bytecode:** an intermediate program produced by the Java compiler. It is not machine code, and can run on any machine that has a JVM.
- **JVM (Java Virtual Machine):** interprets the bytecode and executes the program.
- **JIT (Just-In-Time) compiler:** used by some JVMs to translate bytecode into native machine code immediately before running it, for faster execution.

Because bytecode compiled on one machine can be interpreted on any machine with a JVM, Java is **portable**. This principle is called **Write Once, Run Anywhere (WORA)**.

Q6. What is a hybrid compiler? Explain with the Java example.

A: A hybrid compiler combines compilation and interpretation. Java source code is first compiled into an intermediate form called bytecode, which is then interpreted by the JVM. Some JVMs use a JIT compiler to translate bytecode into machine code right before execution for extra speed.

Q7. Why is Java called portable? What does WORA mean?

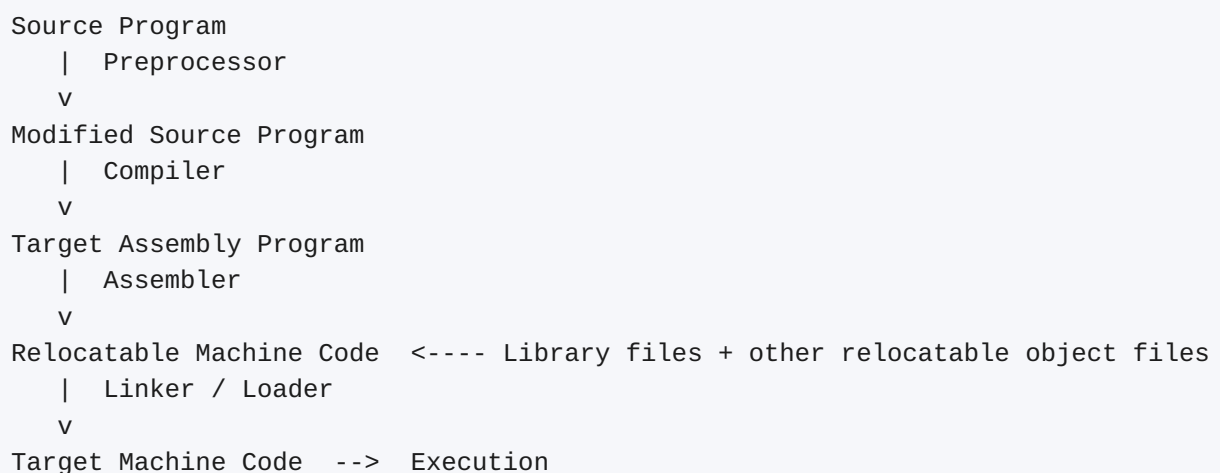
A: Java programs are compiled into bytecode, and the same bytecode can run unchanged on any machine that has a JVM installed. This is summarized as "Write Once, Run Anywhere" (WORA).

Q8. Define bytecode, JVM, and JIT compiler.

A: **Bytecode** is an intermediate program generated by the Java compiler — it is not machine code and can run on any machine with a JVM. The **JVM (Java Virtual Machine)** interprets Java bytecode and executes the program. The **JIT (Just-In-Time) compiler** converts bytecode into machine language immediately before execution, improving program speed.

2.3 Complete Language-Processing System

Besides the compiler, several other programs are usually needed to produce the final executable target program.



- **Preprocessor:** collects source files (if the program is split across modules) and expands **macros** — shorthand instructions — into full source statements. It produces the modified source program fed to the compiler.
- **Compiler:** translates the modified source program into a target assembly program. Assembly output is preferred over raw machine code because assembly is easier to generate and easier to debug.
- **Assembler:** converts the assembly-language program into **relocatable machine code** (machine code whose final memory addresses are not yet fixed).
- **Linker:** combines relocatable object files with library files into one program, resolving external memory addresses (where code in one file refers to a location in another file).
- **Loader:** loads the final executable object files into memory and prepares them for execution.

Q9. What is a preprocessor and what is a macro?

A: A preprocessor prepares the source program before compilation — it collects source files split into modules and expands macros (parameterized shorthand instructions) into complete source-language statements.

Q10. Why do compilers often output assembly language instead of machine language directly? (Exercise 1.1.3)

A: Assembly language is easier to produce as compiler output and easier to debug/read than raw machine code. A separate assembler can then reliably convert it into correct machine code.

Q11. What tasks does an assembler perform? (Exercise 1.1.5)

A: It translates mnemonic assembly instructions into binary machine instructions, and produces relocatable machine code (object code) that can later be linked with other object files and libraries.

Q12. What is relocatable machine code? Define linker and loader.

A: **Relocatable machine code** is machine code whose final memory addresses are not yet fixed. The **linker** combines relocatable object files and library files, resolving external memory addresses (where code in one file refers to a location in another file). The **loader** puts together all the executable object files into memory and prepares them for execution.

Q13. Why is a linker needed?

A: Large programs are often compiled in separate pieces (modules). The linker joins these relocatable object files together with library files into one executable program, resolving addresses that refer across files.

Q14. Describe the complete language-processing system step by step.

A: (1) The programmer writes the source program. (2) The preprocessor expands macros and collects source modules. (3) The compiler translates the modified source into a target assembly program, reporting errors. (4) The assembler converts the assembly program into relocatable machine code. (5) The linker combines relocatable object files and library files, resolving external addresses. (6) The loader loads the completed executable into memory. (7) The CPU executes the program.

Q15. A compiler that translates one high-level language into another is called a source-to-source translator. What are the advantages of using C as the target language? (Exercise 1.1.4)

A: C runs on almost every platform (high portability); mature, highly optimizing C compilers already exist, so efficient machine code is generated without extra effort; it avoids building a new machine-specific code generator from scratch; and existing C tools make debugging and optimization easier.

3. Structure of a Compiler

A compiler is not a single black box. Internally, it maps the source program to the target program through two broad parts — **analysis** and **synthesis** — which are further broken into a sequence of **phases**.

3.1 Analysis and Synthesis (Front End / Back End)

- **Analysis part (Front End):** breaks the source program into pieces, imposes grammatical structure on them, builds an intermediate representation, and creates the symbol table. It reports errors if the program is syntactically ill-formed or semantically unsound.
- **Synthesis part (Back End):** constructs the target program from the intermediate representation and the symbol table.

Front End (Analysis)	Back End (Synthesis)
Analyzes the source program	Synthesizes the target program
Detects syntax and semantic errors	Generates machine code
Builds the intermediate representation	Uses the intermediate representation
Builds the symbol table	Uses the symbol table

Q16. What are the Front End and Back End of a compiler?

A: The **Front End (Analysis)** reads and breaks up the source program, detects syntax and semantic errors, and builds the intermediate representation and symbol table. The **Back End (Synthesis)** uses the intermediate representation and symbol table to generate the target (machine) program.

Q17. Why is a compiler divided into phases?

A: Dividing translation into phases makes the compiler easier to design, understand, debug, maintain, and makes it more efficient overall. Each phase transforms one representation of the program into another.

3.2 Phases of a Compiler

The standard phases, in order, are:

```

Character Stream
  | Lexical Analyzer
  v
Token Stream
  | Syntax Analyzer
  v
Syntax Tree
  | Semantic Analyzer
  v
(Annotated) Syntax Tree
  | Intermediate Code Generator
  v
Intermediate Representation
  | Machine-Independent Code Optimizer (optional)
  v
Intermediate Representation
  | Code Generator
  v
Target-Machine Code
  | Machine-Dependent Code Optimizer (optional)
  v
Target-Machine Code

```

The **Symbol Table** is used by all phases of the compiler, not just one. The two optimization phases are optional — one or both may be missing.

Q18. What is an Intermediate Representation (IR), and why is it needed?

A: An IR is an internal form of the source program created after analysis, acting as a bridge between the source program and the target program. It is needed because it separates the front end from the back end, makes code optimization and code generation easier, and lets one front end be combined with different back ends for different target machines (and vice versa).

3.2.1 Lexical Analysis (Scanning)

The first phase reads the source program as a stream of characters and groups them into meaningful sequences called **lexemes**. For each lexeme it outputs a **token** of the form `<token-name, attribute-value>`, which is passed to the syntax analyzer.

- **token-name**: an abstract symbol used during syntax analysis (e.g. id, +, *).

- **attribute-value:** points to the symbol-table entry for that token; needed later for semantic analysis and code generation.

Spaces, tabs, and blank lines do not change program meaning, so the lexical analyzer discards them.

Q19. What is a lexeme and what is a token?

A: A lexeme is a meaningful sequence of characters in the source program (e.g. "position", "+"). A token is the <token-name, attribute-value> pair produced for each lexeme and passed to the syntax analyzer.

Q20. Why is lexical analysis needed? What is its input and output?

A: Input: character stream (source program). Output: token stream. It is needed because the compiler cannot work directly with raw individual characters — converting them into tokens makes it far easier for later phases to analyze the program grammatically and semantically.

3.2.2 Syntax Analysis (Parsing)

The parser uses the token names to build a tree-like intermediate representation called a **syntax tree**, showing the grammatical structure of the token stream. Each interior node represents an operation, and its children represent the operands.

Q21. What is a syntax tree used for?

A: It represents the grammatical structure and order of operations in a statement, helping later phases (semantic analysis, code generation) understand how operators and operands relate to each other.

Q22. Why is syntax analysis needed? What is its input and output?

A: Input: token stream. Output: syntax tree. After lexical analysis the compiler only has a flat list of tokens — it still does not know whether they are arranged correctly according to the language grammar. Syntax analysis checks grammatical correctness, detects syntax errors, and builds the syntax tree for later phases.

3.2.3 Semantic Analysis

The semantic analyzer checks the syntax tree against the language's rules for semantic consistency (e.g., does every operator have compatible operand types?), using and updating the symbol table. This step is called **type checking**.

Coercion is an implicit type conversion permitted by the language, such as converting an integer to a floating-point number when it is combined with a float in an expression.

Q23. Why is semantic analysis needed?

A: A program can be grammatically correct yet still be semantically meaningless — for example, applying an operator to incompatible types. Semantic analysis is needed to catch such errors and to gather type information for later code generation.

Q24. What is type checking? Give an example of a semantic error.

A: Type checking verifies that operators are applied to compatible operand types. Example: many languages require an array index to be an integer, so `A[2.5]` would be a semantic error.

Q25. What is coercion?

A: Coercion is an automatic type conversion allowed by the language specification — for example, converting an integer operand into a floating-point number so it can be used with a float operand in the same expression.

3.2.4 Intermediate Code Generation

After analysis, the compiler generates a low-level, machine-like intermediate representation — a program for an abstract machine. It must be easy to produce and easy to translate into the target machine. A common form is **three-address code**, a sequence of instructions with at most one operator per instruction and up to three operands.

Q26. Why is intermediate code generation needed?

A: It separates the front end from the back end and makes both optimization and code generation easier. The intermediate representation should have two properties — it should be easy to produce and easy to translate into the target machine.

Q27. What is three-address code?

A: An intermediate representation made of assembly-like instructions, each with at most one operator on the right-hand side and up to three operands. The compiler generates temporary names (t1, t2, ...) to hold intermediate values.

3.2.5 Code Optimization

This optional, machine-independent phase improves the intermediate code so the back end can generate better target code — usually meaning faster code, though it may also target smaller size or lower power use.

3.2.6 Code Generation

The final phase maps the (optimized) intermediate representation into the target language, typically machine code. It selects registers or memory locations for variables. The careful, judicious assignment of registers is a crucial part of this phase.

Storage allocation — deciding where variables are stored at run time (registers or memory) — is decided during intermediate code generation or code generation, depending on the source language.

Q28. Is code optimization compulsory?

A: No. Code optimization is optional — a compiler can produce correct machine code without it. Its only purpose is to improve the quality (speed, size, or power usage) of the generated code.

Q29. What are the advantages of code optimization, and what is an "optimizing compiler"?

A: Advantages: removes unnecessary instructions, improves execution speed, reduces code size and memory usage, and reduces power consumption. An "optimizing compiler" spends significant time on this phase to generate highly efficient target code — the amount of optimization performed varies greatly between compilers.

Q30. Why is code generation needed, and what is storage allocation?

A: Code generation is needed because the computer cannot execute intermediate code directly — it must be mapped into real target-machine instructions. Storage allocation is the decision of where each variable will be stored at run time (in a register or in memory); this decision is made either during intermediate code generation or during code generation.

Q31. Why does register allocation matter in code generation?

A: Registers are much faster to access than memory, so choosing which variables get stored in registers has a large effect on the speed of the generated program. This is called the judicious assignment of registers.

3.3 Symbol Table

The symbol table is a data structure containing one record per variable/procedure name, storing its attributes:

- Storage allocated for the name
- Type
- Scope — where in the program the name may be used
- For procedures — number/type of arguments, how each is passed (by value or reference), and the return type

It must allow the compiler to find and store/retrieve a name's record quickly, since almost every phase uses it.

Q32. What information does a symbol table store, and why is it important?

A: It stores each name's storage location, type, scope, and (for procedures) argument details and return type. It is important because nearly every phase of the compiler — semantic analysis, intermediate code generation, code generation — depends on it.

3.4 Grouping of Phases into Passes

A **phase** is a logical step in compilation. A **pass** is an implementation-level grouping — activities from several phases combined together to read an input file once and write an output file once.

Phase	Pass
A logical step of compilation	An implementation-level grouping of steps
Each phase has one specific job	A pass may combine several phases

A common grouping: front-end phases (lexical, syntax, semantic analysis, intermediate code generation) form one pass; code optimization forms an optional pass; code generation forms the back-end pass. Because of this separation, one front end can be combined with different back ends to support multiple target machines, and different front ends can share one back end to support multiple source languages.

Q33. What is the difference between a phase and a pass?

A: A phase is one logical stage of translation (e.g., lexical analysis). A pass is a grouping of one or more phases that together read an input file and produce an output file in a single implementation step.

3.5 Compiler-Construction Tools

- **Parser generators** — produce syntax analyzers from a grammar description.
- **Scanner generators** — produce lexical analyzers from a regular-expression description of tokens.
- **Syntax-directed translation engines** — produce routines for walking a parse tree and generating intermediate code.
- **Code-generator generators** — produce a code generator from rules mapping intermediate-language operations to target-machine instructions.
- **Data-flow analysis engines** — gather information about how values move through a program; key to optimization.
- **Compiler-construction toolkits** — integrated routines for building various compiler phases.

Q34. Why are compiler-construction tools needed?

A: Besides general software-development tools (editors, debuggers, version managers, profilers), specialized tools help implement specific compiler phases. They reduce development time, automate compiler construction, hide complex implementation details, and produce components that integrate easily into the rest of the compiler.

4. Simulation: Translating an Assignment Statement

Trace the statement below through every phase of the compiler:

```
position = initial + rate * 60
```

Step 1 — Lexical Analysis (Input: character stream)

Lexemes: `position`, `=`, `initial`, `+`, `rate`, `*`, `60` (blanks discarded).

```
<id,1> <=> <id,2> <+> <id,3> <*> <60>
```

Symbol table so far: 1 = position, 2 = initial, 3 = rate.

Step 2 — Syntax Analysis (Output: syntax tree)

The parser builds a tree showing that `rate * 60` is computed first (higher precedence), then added to `initial`, and the result assigned to `position`.

```

      =
     / \
  <id,1> +
       / \
    <id,2> *
         / \
      <id,3> 60
  
```

Step 3 — Semantic Analysis (Output: annotated syntax tree)

Assume position, initial, and rate are declared as float, but the lexeme 60 is an integer. The type checker finds that `*` is applied to a float (rate) and an integer (60), so it inserts an explicit **inttofloat** conversion node around 60 (coercion).

Step 4 — Intermediate Code Generation (Output: three-address code)

```
t1 = inttofloat(60)
t2 = id3 * t1
t3 = id2 + t2
id1 = t3
```

Step 5 — Code Optimization (Output: optimized three-address code)

The optimizer converts `60` to `60.0` once at compile time (removing the `inttofloat` operation), and eliminates `t3` since it is used only once.

```
t1 = id3 * 60.0
id1 = id2 + t1
```

Step 6 — Code Generation (Output: target machine code)

```
LDF  R2, id3          ; load id3 (rate) into R2
MULF R2, R2, #60.0    ; multiply R2 by immediate constant 60.0
LDF  R1, id2          ; load id2 (initial) into R1
ADDF  R1, R1, R2       ; add R2 to R1
STF  id1, R1          ; store result into id1 (position)
```

The first operand of each instruction is the destination; "F" marks a floating-point instruction; "#" marks an immediate constant.

Q35. Trace $\text{position} = \text{initial} + \text{rate} * 60$ through all six phases (expect this in exams).

A: Lexical analysis produces the token stream $\langle \text{id}, 1 \rangle \langle = \rangle \langle \text{id}, 2 \rangle \langle + \rangle \langle \text{id}, 3 \rangle \langle * \rangle \langle 60 \rangle$. Syntax analysis builds a tree showing $\text{rate} * 60$ evaluated before the addition. Semantic analysis inserts an `inttofloat` node to coerce 60 to a float. Intermediate code generation produces four three-address instructions using temporaries `t1–t3`. Code optimization removes the `inttofloat` step and the redundant `t3`, leaving two instructions. Code generation finally emits five floating-point machine instructions using registers `R1` and `R2`.

5. Errors in Compilation

Even if a compiler produces an executable that behaves exactly like the source program, this does not guarantee the program is logically correct — it may still crash at run time due to memory faults, file I/O errors, etc. **Compilation errors** are strictly the errors identified during the **analysis phase**; the synthesis phase never reports an error.

A compiler throws a compilation error when it cannot identify the intended token, finds a grammatical (syntax) mistake, or cannot determine a valid meaning for an expression or statement. In each case, it becomes impossible to generate the corresponding machine code.

Q36. What are compilation errors, and which phase reports them?

A: Compilation errors are errors identified during the analysis phase (lexical, syntax, or semantic analysis) — the synthesis (back end) phase never reports an error, since by the time it runs the program has already been validated.

Q37. Does an error-free compilation guarantee a correct program?

A: No. The compiler only guarantees the generated executable behaves the same as the source program. It cannot guarantee the program's logic is correct — the program may still crash from run-time issues such as memory faults or file read/write errors.

6. Evolution of Programming Languages

Early computers (1940s) were programmed directly in machine language — sequences of 0s and 1s specifying very low-level operations. This was slow, tedious, and error-prone, motivating the development of higher-level languages.

The first step was **mnemonic assembly languages** (early 1950s), where instructions were symbolic representations of machine instructions. Later, **macro instructions** let programmers define shorthand for frequently used instruction sequences.

A major step came in the late 1950s with **Fortran** (scientific computation), **Cobol** (business data processing), and **Lisp** (symbolic computation) — higher-level notations that are still used today.

6.1 Generations of Languages

Generation	Description	Examples
1GL	Machine language (0s and 1s)	Raw binary code
2GL	Assembly language	Mnemonic instructions
3GL	Higher-level languages	Fortran, Cobol, Lisp, C, C++, C#, Java
4GL	Application-specific languages	NOMAD (reports), SQL (databases), Postscript (text formatting)
5GL	Logic / constraint-based languages	Prolog, OPS5

6.2 Classification of Languages

Imperative	Declarative
Specifies <i>how</i> a computation is done	Specifies <i>what</i> computation is done
Has program state and statements that change it	Focuses on the desired result, not the steps
C, C++, C#, Java	ML, Haskell (functional); Prolog (constraint logic)

- **von Neumann language:** computational model based on the von Neumann architecture (e.g. Fortran, C).
- **Object-oriented language:** supports OOP — programs made of interacting objects. Simula 67 and Smalltalk were early examples; C++, C#, Java, Ruby are modern ones.
- **Scripting language:** interpreted languages with high-level operators for "gluing together" computations (originally called scripts). Examples: Awk, JavaScript, Perl, PHP, Python, Ruby, Tcl. Scripts are usually much shorter than equivalent C programs.

Exercise 1.3.1 — Classify Common Languages

Language	Applicable Terms
C	Imperative, von Neumann, Third-generation
C++	Imperative, von Neumann, Object-oriented, Third-generation
Cobol	Imperative, von Neumann, Third-generation
Fortran	Imperative, von Neumann, Third-generation
Java	Imperative, Object-oriented, Third-generation
Lisp	Third-generation
ML	Declarative, Functional
Perl	Scripting
Python	Object-oriented, Scripting
VB	Imperative, Object-oriented, Third-generation

Q38. Differentiate imperative and declarative languages.

A: Imperative languages specify how a computation should be carried out step by step and rely on a changing program state (e.g. C, Java). Declarative languages specify what result is wanted without detailing the steps (e.g. Prolog, Haskell).

6.3 Impact on Compilers

Since programming languages and compilers are closely linked, every new language feature forces compiler writers to devise new translation algorithms; every advance in computer architecture forces compilers to exploit the new hardware capabilities.

Compiler writing is challenging because a compiler is a large program, must handle a potentially infinite set of possible source programs, and the problem of generating truly optimal code is **undecidable** in general — so compiler writers must make tradeoffs. It is rewarding because a compiler writer's decisions affect every program the compiler compiles.

Q39. Why is compiler writing considered both challenging and rewarding?

A: It is challenging because a compiler is a large, complex program that must correctly handle an infinite set of possible source programs, and generating truly optimal code is undecidable. It is rewarding because a compiler writer's work affects every program that the compiler will ever compile.

7. The Science of Building a Compiler

Compiler design solves complex real-world translation problems by abstracting them into mathematical models, then solving the models with rigorous techniques — validated and refined through experimentation. A compiler must preserve the exact meaning of every source program it translates, no matter how large or complex.

7.1 Modeling in Compiler Design

Model	Used For
Finite-State Machines	Recognizing lexical units (keywords, identifiers, tokens)
Regular Expressions	Describing the tokens of a language
Context-Free Grammars	Describing syntactic structure (e.g. nested parentheses, control constructs)
Trees	Representing program structure and its translation into object code
Graphs, Matrices, Linear Programs	Producing well-optimized code

Q40. What is the main goal of modeling in compiler design?

A: To choose the right mathematical model and algorithm for each translation problem, balancing generality/power against simplicity/efficiency.

7.2 The Science of Code Optimization

"Optimization" is technically a misnomer — no compiler can guarantee its output code is the fastest possible for a given task; it can only try to improve on the obvious/naïve translation. Optimization has become more important (parallel/multicore machines lose performance by orders of magnitude without it) and more complex (modern processor architectures offer more opportunities to improve execution) over time.

Design Objectives of Code Optimization

1. **Correctness** — must preserve the meaning of the compiled program (the most important objective).
2. **Performance** — must improve performance for many programs, not just a few cases.
3. **Reasonable compilation time** — to support a rapid development/debugging cycle.
4. **Manageable engineering effort** — implementation effort must remain practical.

Correctness comes first because it is trivial to write a compiler that produces fast but incorrect code. Programs are often first developed and debugged without optimization, since unoptimized code keeps a clear relationship between source and object code, making debugging easier; testing is then repeated once optimizations are enabled.

Q41. Why is "optimization" considered a misnomer?

A: Because no compiler can guarantee its generated code is truly the fastest or best possible for a task — it can only try to produce code that is better than the naive/obvious translation.

Q42. Why can't a compiler rely only on "hacks"? Why is pure theory also insufficient?

A: It is hard to build a robust compiler out of ad-hoc hacks, so an extensive, rigorous mathematical theory has been built around optimizing code, letting us prove an optimization is correct for all possible inputs. But pure theory alone is insufficient because most compiler-optimization questions are undecidable — good understanding of real program behavior plus thorough experimentation is also required.

Q43. List the four design objectives of code optimization, and state which is most important.

A: (1) Correctness, (2) Performance improvement, (3) Reasonable compilation time, (4) Manageable engineering effort. Correctness is the most important, since generating fast but incorrect code is worthless.

Q44. Why are unoptimized programs easier to debug?

A: Because optimizations often obscure the relationship between the source code and the generated object code. Without optimization, that relationship stays clear, making errors easier to trace back to the source.

8. Applications of Compiler Technology

Compiler technology is useful beyond just building compilers — it influences language design, hardware design, and many software tools.

8.1 Implementation of High-Level Programming Languages

High-Level Language	Low-Level Language
Easier to program, read, debug, maintain	Harder to write, read, debug, maintain
More portable, less error-prone	Less portable, more error-prone
Target programs generally run slower	Programmer has more control; can be more efficient

Optimizing compilers exist precisely to close this performance gap.

Example: the C `register` keyword

When C was created in the 1970s, the `register` keyword let programmers manually choose which variables should live in registers. Once effective automatic register-allocation techniques were developed, this control became unnecessary — and can even hurt performance today, since programmers are usually not the best judge of low-level, machine-specific decisions like register allocation.

Language popularity has shifted toward higher abstraction over time: C dominated the 1980s, C++ rose in the 1990s, Java (1995) gained popularity in the late 1990s. Each shift spurred new compiler-optimization research:

- **Data-flow optimizations** — remove redundancy across high-level constructs like arrays, structures, loops, and procedure calls.

- **Procedure inlining** — replaces a procedure call with the procedure's body, useful for object-oriented code with many small methods.
- **Virtual method dispatch optimization** — speeds up OOP method calls.

Java's safety features (type safety, array bounds checking, no pointers, automatic garbage collection) make programming easier but add run-time overhead. Compilers reduce this overhead by eliminating unnecessary range checks and allocating short-lived objects on the stack instead of the heap.

Dynamic compilation compiles bytecode into native code at run time (as Java does). **Dynamic optimization** more broadly uses run-time information to produce better-optimized code — typically by compiling and optimizing only the frequently executed parts of a program, to keep compilation overhead low.

Q45. What are the two key ideas behind Object-Oriented Programming, and what optimizations does OOP demand?

A: The two key ideas are (1) data abstraction and (2) inheritance of properties, both of which make programs more modular and easier to maintain. Because OOP programs use many small procedures (methods), compiler optimizations must work well across procedure boundaries — chiefly procedure inlining (replacing a call with the procedure's body) and optimizations to speed up virtual method dispatch.

Q46. Why is the `register` keyword in C less useful today?

A: Modern compilers use effective automatic register-allocation algorithms that outperform manual choices, since the best allocation depends on the specific machine architecture. Manual register hints can even hurt performance on a different machine than the one the program was written for.

Q47. What is dynamic compilation, and what is dynamic optimization?

A: Dynamic compilation converts bytecode to native machine code at run time (e.g. Java's JIT). Dynamic optimization uses run-time information more generally to improve performance, typically by optimizing only the frequently executed parts of a program to keep overhead low.

8.2 Optimizations for Computer Architectures

High-performance systems rely on two techniques: **parallelism** and **memory hierarchies**.

Parallelism

Instruction-level parallelism (multiple operations executed simultaneously) is exploited by all modern processors, often hidden from the programmer. Compilers can rearrange instructions to make this parallelism more effective. Some instruction sets expose parallelism explicitly — **VLIW** (Very Long Instruction Word) machines like the Intel IA64 issue multiple operations per instruction. At the processor level, multiple threads run on different processors (multiprocessors); compilers can automatically generate parallel code from sequential programs.

Memory Hierarchies

A memory hierarchy has several levels of storage with different speed/size tradeoffs — the level closest to the processor is fastest but smallest.

```
Registers (fastest, smallest, hundreds of bytes)
-> Caches (kilobytes to megabytes)
-> Physical memory (megabytes to gigabytes)
-> Secondary storage (gigabytes and beyond, slowest, largest)
```

Using registers effectively is probably the single most important optimization problem, since (unlike caches and memory) registers must be managed explicitly by the compiler. Compilers also improve cache effectiveness by changing data layout, instruction order, and code layout.

Q48. What are the two basic techniques that high-performance systems rely on?

A: Parallelism (instruction-level and processor-level) and memory hierarchies (multiple levels of storage with different speed/size tradeoffs).

Q49. Why is effective register usage considered the most important optimization problem?

A: Because registers, unlike caches and main memory, are not managed automatically by hardware — the compiler must explicitly decide which values to keep in registers, and this decision greatly affects program speed.

8.3 Design of New Computer Architectures

Modern computer architectures are designed together with their compilers — compiled code running on simulators is used to evaluate proposed hardware features before the hardware is even built.

RISC vs CISC

RISC	CISC
Reduced Instruction-Set Computer	Complex Instruction-Set Computer
Small number of simple instructions	Many complex instructions
Easier for the compiler to optimize	Harder for the compiler to optimize
Simpler hardware	More complex hardware
PowerPC, SPARC, MIPS, Alpha, PA-RISC	x86

Compiler optimizations can often reduce complex CISC instructions to a few simpler operations, which is why simple RISC instruction sets are preferred — compilers can use them effectively and hardware is easier to optimize. Although x86 uses a CISC instruction set, many RISC ideas are used inside its own implementation.

Specialized Architectures

Concepts such as data-flow machines, vector machines, VLIW machines, SIMD processor arrays, systolic arrays, and shared/distributed-memory multiprocessors each required matching compiler technology. Since embedded systems can fit entire processors on one chip, they can be tailored to specific applications rather than using general-purpose designs, resulting in a wide diversity of architectures — compilers are needed both to program these chips and to evaluate their designs.

Q50. Compare RISC and CISC, and explain why compilers favor simple instruction sets.

A: RISC uses a small set of simple instructions that are easy for compilers to use effectively and for hardware to optimize; CISC uses many complex instructions. Compiler optimizations can reduce complex instructions to a handful of simpler operations, so a simple instruction set lets the compiler generate efficient code more easily.

8.4 Program Translations

Compiling technology can translate between many kinds of languages/representations, not only from high-level code to machine code.

- **Binary translation** — translates binary code from one instruction set to another, letting a machine run programs compiled for a different architecture (e.g. converting x86 code to Alpha/Sparc code; Transmeta Crusoe's VLIW processor converting x86 code to native VLIW code; Apple's MC68040-to-PowerPC transition in 1994).
- **Hardware synthesis** — translates hardware description languages (Verilog, VHDL) written at the register-transfer level (RTL) into gates and eventually a physical chip layout. Unlike software compilers, these tools may take hours to optimize a circuit.
- **Database query interpreters** — query languages such as SQL are interpreted or compiled into commands that search a database for records matching a predicate.
- **Compiled simulation** — instead of interpreting a hardware design (e.g. in Verilog/VHDL) at simulation time, the design is compiled into native machine code, running orders of magnitude faster than an interpreter-based simulator — useful since simulation of many design alternatives can otherwise take days.

Q51. What is binary translation, and give one real example.

A: Binary translation converts the binary code compiled for one machine's instruction set into that of another, allowing programs to run on different hardware. Example: Apple used binary translation to let PowerPC-based Macintoshes (1994) run legacy Motorola MC68040 code.

Q52. What is hardware synthesis?

A: Even hardware designs are mostly written in high-level hardware description languages like Verilog and VHDL, typically at the register-transfer level (RTL), where variables represent registers and expressions represent combinational logic. Hardware-synthesis tools automatically translate RTL descriptions into gates, which are then mapped to transistors and a physical layout — these tools may take hours optimizing a circuit.

Q53. What are database query interpreters?

A: Query languages, especially SQL, are used to search databases. Database queries consist of predicates containing relational and boolean operators; they can be interpreted, or compiled into commands that search a database for records satisfying that predicate.

Q54. Why is compiled simulation faster than an interpreter-based simulator?

A: Because the hardware design is compiled once into native machine code that simulates it directly, instead of an interpreter re-reading and re-interpreting the design on every simulation run — giving orders-of-magnitude speedup, important since many design alternatives must be tested.

8.5 Software Productivity Tools

Programs are complex, so errors are common. Testing is the primary way to find them, but **data-flow analysis** — a technique originally developed for compiler optimization — can find errors **statically** (before the program runs), across all possible execution paths rather than just the paths a test happens to exercise.

Because finding all program errors is undecidable, practical error detectors are usually **unsound or incomplete** — they may miss real errors, and not every warning is a genuine error. This differs from compiler optimizations, which must always be conservative and can never change a program's meaning.

Type Checking & Security

Type checking catches inconsistencies such as mismatched operand types or wrong procedure arguments, and can be extended to flow analysis (e.g. detecting a null pointer dereferenced immediately after assignment). The same idea can label user-supplied strings as "dangerous" to catch security holes if such data is never validated before it influences program control flow.

Bounds Checking

Low-level languages like C have no automatic array bounds checking, which is a major cause of **buffer overflow** security vulnerabilities. The same data-flow analysis used to remove redundant range checks can also be used to detect potential buffer overflows — though failing to identify a genuine overflow is far more serious than failing to optimize away a safe redundant check.

Memory-Management Tools

Garbage collection automatically frees memory no longer in use, eliminating memory leaks common in C/C++. Tools such as **Purify** dynamically catch memory-management errors as a program runs; other tools detect such problems statically.

Q55. What is type checking, and how can the same technology catch security holes?

A: Type checking is an effective, well-established technique that catches inconsistencies, such as an operation applied to the wrong type of object or a pointer that is assigned null and then immediately dereferenced. The same technology can catch security holes: a user-supplied string can be labeled with a type such as "dangerous". If it is never checked for proper format and is later able to influence the program's control flow, there is a potential security flaw.

Q56. Why are practical error detectors often "unsound" or "incomplete"? How does this differ from compiler optimizations?

A: Finding all program errors is undecidable, so error detectors may miss real errors (incomplete) or flag false alarms (unsound) — otherwise users would be overwhelmed and stop using the tool. Compiler optimizations, in contrast, must always be conservative and can never change program meaning under any circumstance.

Q57. Why are buffer overflows a serious problem in C, and how can compiler techniques help detect them?

A: C provides no automatic array bounds checking, so a program can write user-supplied data past the end of a buffer, letting an attacker corrupt memory and compromise the system. Data-flow analysis, similar to that used for optimizing range checks, can be used to statically detect potential buffer overflows.

Q58. What is a memory leak, and how do tools like Purify help?

A: A memory leak occurs when a program allocates memory but never frees it after it is no longer needed, wasting memory over time. Purify is a tool that dynamically detects such memory-management errors while the program runs; other tools detect similar problems statically.

9. Extra: Types of Compilers and Interpreters

Supplementary terms useful for exam recognition.

9.1 Types of Compilers

Type	Description
Cross-compiler	Runs on one machine/OS but produces code for a different machine/OS.
Native compiler	Produces output that runs on the same machine/OS as the compiler itself.
Bootstrap compiler	Written in the same language it compiles.
Decompiler	Translates code from a low-level language back into a higher-level one.
Source-to-source compiler (Transpiler)	Translates between two high-level languages (e.g. Babel: ES6+ to ES5; Emscripten: C/C++ to JavaScript).
Bytecode compiler	Translates a high-level language into an intermediate language interpreted by a virtual machine (e.g. Java, Python).
Just-In-Time (JIT) compiler	Defers compilation until run time; usually runs inside an interpreter (e.g. Java, modern JavaScript engines).
Ahead-of-Time (AOT) compiler	Compiles a high-level language (or bytecode) before run time (e.g. Angular compiling TypeScript to JavaScript at build time).
Assembler	Translates human-readable assembly language into machine code.

9.2 Types of Interpreters

Type	Description
Bytecode interpreter	Executes instructions represented as bytes; used by languages like Python and Java before JIT compilation.
Threaded code interpreter	Uses pointers instead of bytes for each instruction (e.g. Forth code in Open Firmware systems).
Abstract Syntax Tree (AST) interpreter	Executes a program by walking its syntax tree directly; simpler to build but slower than bytecode interpretation.

The line between compilers and interpreters is often blurred: **JIT compilation** combines both ideas by compiling code to native machine instructions at run time, inside what is otherwise an interpreter-driven system.

Q59. What is a transpiler? Give two examples.

A: A transpiler (source-to-source compiler) translates code from one high-level language into another high-level language, rather than into machine code. Examples: Babel (ES6+ JavaScript to ES5) and Emscripten (C/C++ to JavaScript).

Q60. Differentiate AOT and JIT compilation.

A: AOT (Ahead-of-Time) compilation happens before the program runs, producing native code in advance. JIT (Just-In-Time) compilation happens during execution, compiling code (often bytecode) into native machine instructions right before it is needed.

Q61. What is the difference between a bytecode interpreter and an AST interpreter?

A: A bytecode interpreter first converts source code into a compact, byte-based instruction set and then executes those bytes, which is generally faster. An AST interpreter walks the syntax tree directly and executes it node by node, which is simpler to build but runs slower due to extra tree-traversal overhead.

Q62. Why do modern JavaScript engines use JIT compilation instead of pure interpretation?

A: Early JavaScript engines were simple interpreters and were slow for compute-heavy code. Modern engines use JIT compilation to translate frequently executed (hot) code paths into native machine code at run time, combining the portability of an interpreted language with execution speed close to a compiled one.

9.3 Quick Comparison

Term	Key Idea
Compiler	Translates the whole program before execution
Interpreter	Executes the program statement by statement, no separate output file
Transpiler	Translates between two high-level languages
JIT Compiler	Compiles to native code at run time, usually inside an interpreter
AOT Compiler	Compiles to native code before run time

Q63. What is the difference between a decompiler and an assembler/disassembler?

A: A decompiler translates low-level code back into a higher-level language. An assembler translates assembly language into machine code, while its inverse, a disassembler, converts machine code back into assembly language.

Q64. Why is a bootstrap compiler significant?

A: A bootstrap compiler is written in the very language it compiles. It is significant because it shows the language is mature enough to build its own tools, and it is commonly used to create the first working compiler for a brand-new language, which is then used to compile improved versions of itself.